LFM 2.5: The Evolution of Liquid Neural Networks

Research Notes

March 2026

LFM 2.5: The Evolution of Liquid Neural Networks

1. Historical Timeline

The Liquid Foundation Model (LFM) 2.5 family represents the culmination of a research thread spanning five years, originating in biologically inspired continuous-time neural networks and converging with modern deep signal processing:

Year	Milestone	Key Reference
2020	Neural Circuit Policies (NCP)	Lechner et al., Nature Machine Intelligence
2021	Liquid Time-Constant (LTC) Networks	Hasani et al., AAAI 2021
2022	Closed-form Continuous-depth (CfC)	Hasani et al., Nature Machine Intelligence
2022	S4 Structured State Spaces	Gu, Goel, R'e, ICLR 2022
2023	Liquid-S4	Hasani et al., ICLR 2023
2023	Hyena Operators	Poli, Massaroli et al., ICML 2023
2023	StripedHyena	Together AI / Liquid AI
2024	LFM 1.0	Liquid AI, October 2024
2025	LFM2 Technical Report	arXiv:2511.23404, November 2025
2025	STAR Architecture Search	ICLR 2025 (Oral)
2026	**LFM 2.5**	Liquid AI, January 2026

Key researchers: Ramin Hasani (CEO, Liquid AI), Mathias Lechner, Alexander Amini, Daniela Rus (MIT CSAIL). Liquid AI is an MIT CSAIL spinoff founded to commercialize these ideas.

---

2. Liquid Time-Constant (LTC) Networks — The Foundation

2.1 Biological Motivation

LTC networks are inspired by the nervous system of *C. elegans*, a 302-neuron nematode whose complete connectome is known. The key insight: biological neurons do not operate with fixed time constants. Instead, their integration windows adapt in real time to the statistics of incoming signals.

2.2 The Leaky Neuron ODE

The starting point is a standard leaky integrator model for a non-spiking neuron's membrane potential $v(t)$:

$$\frac{dv}{dt} = -g_L\, v(t) + S(t)$$

where g_L is a fixed leakage conductance and $S(t)$ is the total synaptic input current. This is a simple exponential decay toward a resting potential, driven by synaptic input.

Intuition: Think of $v(t)$ as the "charge" on a leaky capacitor. g_L controls how fast the charge leaks away. $S(t)$ is the current flowing in from other neurons.

2.3 The LTC Equation

Hasani et al. (AAAI 2021) generalize this to the LTC equation by making the effective time constant depend on both the hidden state and the input. For hidden state $x(t)$ with input $I(t)$:

$$\frac{dx(t)}{dt} = -\frac{x(t)}{\tau(x, I)} + \frac{1}{\tau(x, I)} f\big(x(t),\, I(t),\, \theta\big)$$

where:

- $x(t) \in \mathbb{R}^d$ is the hidden state vector
- $I(t)$ is the external input at time t
- $\tau(x, I) > 0$ is the **liquid time constant** — it varies continuously as a function of the current state and input
- $f(\cdot)$ is a learned nonlinear function (typically a shallow neural network with sigmoid or tanh activations) parameterized by θ
- The ratio $1/\tau$ controls both the decay rate and the drive strength

This can be rewritten in the more compact form used in the technical literature:

$$\frac{dx}{dt} = -\big[w + f_\theta(I(t))\big] \cdot x(t) + A \cdot \big[w + f_\theta(I(t))\big]$$

where $w > 0$ is a base decay weight, A is a bias (target equilibrium), and $f_\theta(I)$ is a neural network mapping from input to an input-dependent modulation of the effective time constant.

Intuition: The "liquid" part means τ is not a fixed hyperparameter — it is a learned, input-dependent quantity. When the input is complex or changing rapidly, τ shrinks (the neuron becomes more responsive). When the input is stable, τ grows (the neuron integrates over a longer window). This is analogous to a human reader adjusting their reading speed based on text difficulty.

2.4 Synaptic Nonlinearity

The synaptic input $S(t)$ in LTCs uses a **conductance-based** model with interlinked nonlinear gates:

$$S_j(t) = \sum_i w_{ij}\, \sigma\big(x_i(t)\big)\, \big(A_{ij} - x_j(t)\big)$$

where σ is a bounded, monotonically increasing sigmoidal nonlinearity, w_{ij} are synaptic weights, and A_{ij} are reversal potentials. The term $(A_{ij} - x_j(t))$ provides a bilinear interaction between presynaptic activity and the postsynaptic state, inspired by conductance-based synaptic transmission in real neurons.

Intuition: The driving force on neuron j depends not just on the input weight but on the *difference* between the reversal potential A_{ij} and the current state x_j. This naturally bounds the dynamics and prevents runaway activation.

2.5 Properties

- **Stability**: The dynamics are provably bounded due to the conductance-based formulation.
- **Expressivity**: LTCs are strictly more expressive than standard neural ODEs with fixed time constants.
- **Recurrent**: LTCs are **continuous-time recurrent neural networks**. They must be integrated forward in time using an ODE solver (e.g., Runge-Kutta).

---

3. Neural Circuit Policies (NCPs) — Sparse Wiring

NCPs (Lechner et al., 2020) apply a biologically inspired **sparse wiring topology** to LTC neurons. Rather than fully connected layers, NCPs use a four-layer architecture inspired by *C. elegans*:

1. **Sensory neurons** — receive raw input
2. **Interneurons** — process with lateral connections
3. **Command neurons** — integrate and decide
4. **Motor neurons** — produce output

The connectivity is sparse (up to 90% sparsity in intermediate layers), with specific motifs: feedforward chains, lateral inhibition loops, and small recurrent triads. This yields models with remarkably few parameters (e.g., 19 neurons controlling an autonomous vehicle) that remain interpretable and auditable.

Relevance to LFM 2.5: NCPs demonstrated that liquid-style dynamics could work with extreme sparsity — a principle that later informed the efficient architectures used in LFMs.

---

4. Closed-form Continuous-depth (CfC) Networks

4.1 The Problem with ODE Solvers

LTC networks require numerical ODE integration at every forward pass. For a Runge-Kutta solver, this means multiple function evaluations per time step. This is accurate but **computationally expensive** — a bottleneck for scaling to large models or long sequences.

4.2 The CfC Solution

Hasani et al. (Nature Machine Intelligence, 2022) derived an **approximate closed-form solution** for the LTC ODE, eliminating numerical integration entirely:

$$x(t) = \big(x(0) - A\big)\, \exp\Big(-\big[1 + f_\theta(I(t))\big]\, t\Big) + A$$

where:

- $x(0)$ is the initial hidden state
- A is the equilibrium (bias) vector
- $f_\theta(I(t))$ is a learned nonlinear function of the input
- The exponential decay rate $[1 + f_\theta(I(t))]$ is input-dependent

Intuition: Instead of numerically integrating the ODE step by step, CfC directly computes where the state will be at time t. The solution is an exponential interpolation between the initial state $x(0)$ and the equilibrium A, with the interpolation speed controlled by the input. Larger $f_\theta(I)$ means faster convergence to A; smaller means the state retains more memory of $x(0)$.

4.3 Sigmoid Gating for Gradient Stability

The raw exponential $\exp(-[\cdots]t)$ can cause gradient vanishing for large t. CfC replaces this with a **sigmoid gate**:

$$x(t) = \sigma\big(g_\theta(I,t)\big) \odot f_1(I, x_0) + \big(1 - \sigma(g_\theta(I,t))\big) \odot f_2(I, x_0)$$

where σ is the logistic sigmoid, g_θ is a learned time-dependent gate, f_1 and f_2 are learned state mappings, and $\odot$ is element-wise multiplication. This is a **sigmoid interpolation** between two learned representations, gated by a time- and input-dependent signal.

Intuition: The sigmoid smoothly blends between "remembering" (f_1, analogous to the initial state) and "updating" (f_2, analogous to the equilibrium). This is structurally similar to the gating in GRUs/LSTMs but derived from continuous-time ODE principles rather than heuristic design.

4.4 Speed and Properties

- **1–5 orders of magnitude faster** than ODE-based LTC networks
- Retains input-dependent adaptive time constants
- Can be used as a **drop-in replacement** for LSTMs, GRUs, or ODE-RNNs
- Still a **recurrent** model at this stage (processes sequences step by step)

---

5. Structured State Space Models: S4, Liquid-S4, and Mamba

5.1 S4: State Spaces for Sequences

Gu, Goel, and R'e (ICLR 2022) introduced S4, which casts sequence modeling as a continuous-time state space model (SSM):

Continuous-time form:

$$x'(t) = \mathbf{A}\, x(t) + \mathbf{B}\, u(t)$$
$$y(t) = \mathbf{C}\, x(t) + \mathbf{D}\, u(t)$$

where:

- $x(t) \in \mathbb{R}^n$ is the latent state
- $u(t) \in \mathbb{R}$ is the input signal
- $y(t) \in \mathbb{R}$ is the output
- $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the state transition matrix (initialized with the HiPPO matrix for long-range memory)
- $\mathbf{B} \in \mathbb{R}^{n \times 1}$ is the input projection
- $\mathbf{C} \in \mathbb{R}^{1 \times n}$ is the output projection
- $\mathbf{D}$ is a skip connection (often zero)

Discretization (bilinear transform with step size Δ):

$$\bar{\mathbf{A}} = \left(\mathbf{I} - \tfrac{\Delta}{2}\mathbf{A}\right)^{-1}\left(\mathbf{I} + \tfrac{\Delta}{2}\mathbf{A}\right)$$
$$\bar{\mathbf{B}} = \left(\mathbf{I} - \tfrac{\Delta}{2}\mathbf{A}\right)^{-1}\Delta\, \mathbf{B}$$

This yields a **discrete recurrence**:

$$x_k = \bar{\mathbf{A}}\, x_{k-1} + \bar{\mathbf{B}}\, u_k$$
$$y_k = \mathbf{C}\, x_k$$

Convolutional view: The entire discrete recurrence unrolls into a 1D convolution with kernel $\bar{K} \in \mathbb{R}^L$:

$$\bar{K}_t = \mathbf{C}\, \bar{\mathbf{A}}^t\, \bar{\mathbf{B}}, \qquad y = \bar{K} * u$$

Intuition: S4's power comes from this **dual view** — it can be computed as a recurrence (for autoregressive inference) or as a convolution (for parallel training). The HiPPO initialization of $\mathbf{A}$ gives the state a principled ability to compress and recall long input histories.

5.2 Liquid-S4: Input-Dependent State Transitions

Hasani et al. (ICLR 2023) merged liquid time-constant dynamics with S4. The key modification: make the state transition matrix **A depend on the input**:

$$x'(t) = \mathbf{A}(u(t))\, x(t) + \mathbf{B}\, u(t)$$

where $\mathbf{A}(u) = \mathbf{A}_0 + \Delta\mathbf{A}(u)$ and $\Delta\mathbf{A}$ is a learned function of the input. This is the "liquid" part — the state dynamics adapt to each input in the sequence. The $\mathbf{A}$ matrix uses a diagonal-plus-low-rank (DPLR) decomposition for computational efficiency.

Results: Liquid-S4 achieved state-of-the-art on the Long Range Arena benchmark (87.32% average) and Speech Commands (96.78% accuracy) with 30% fewer parameters than S4.

Intuition: Standard S4 uses the same transition dynamics regardless of what the input looks like. Liquid-S4 says "the way I store and transform information should depend on what I'm currently seeing." This is the same principle as the liquid time constant, now applied to the structured state space framework.

5.3 Mamba: Selective State Spaces (S6)

Gu and Dao (2023) took a similar idea further with Mamba. The **S6** (Selective State Space) layer makes $\mathbf{B}$, $\mathbf{C}$, and the step size Δ all functions of the input:

$$\mathbf{B}_k = f_B(u_k), \quad \mathbf{C}_k = f_C(u_k), \quad \Delta_k = \mathrm{softplus}\big(f_\Delta(u_k)\big)$$

This **selection mechanism** lets the model dynamically decide what to remember and what to forget at each time step — content-aware reasoning rather than fixed-filter processing.

Relevance: Mamba and Liquid-S4 represent converging ideas: input-dependent state space dynamics. Both influenced the design philosophy behind LFM architectures.

---

6. Hyena Operators and StripedHyena

6.1 The Hyena Operator

Poli, Massaroli et al. (ICML 2023) proposed Hyena as a subquadratic replacement for self-attention. The core idea: replace the $O(L^2)$ attention matrix with a recurrence of **implicit long convolutions** interleaved with **data-controlled gating**.

For an order-N Hyena operator, given input u:

1. Compute $N+1$ linear projections: $v, x_1, x_2, \ldots, x_N = \text{proj}(u)$
2. Apply the recurrence:

$$y_0 = v$$
$$y_i = (y_{i-1} * h_i) \odot x_i, \quad i = 1, \ldots, N$$

where $*$ denotes (long) convolution and $\odot$ is element-wise multiplication.

The convolution filters h_i are not stored explicitly — they are generated by a **feed-forward network** applied to positional encodings (hence "implicit" convolution).

Intuition: Each layer of the recurrence refines the representation by (a) convolving with a learned long-range filter (capturing global context) and then (b) multiplying element-wise with a data-dependent projection (steering the computation based on what the input says). This interleaving of convolution and gating replaces the quadratic all-pairs comparison of attention with $O(L \log L)$ operations.

6.2 StripedHyena: The Hybrid Architecture

StripedHyena (Together AI / Liquid AI, late 2023) is a hybrid architecture that **alternates** Hyena-style gated convolution blocks with standard grouped-query attention (GQA) blocks:

$$\text{Layer}_i = \begin{cases} \text{GQA Block} & \text{if } i \bmod 4 = 0 \\ \text{Gated Hyena Convolution} & \text{otherwise} \end{cases}$$

The ratio is typically 1:3 (attention to convolution), though this varies by model size.

Intuition: Attention excels at arbitrary long-range token interactions but is expensive. Convolutions handle local and medium-range patterns efficiently. By using convolution for most layers and reserving attention for periodic "global sync" layers, StripedHyena gets the best of both worlds at lower compute cost.

---

7. LFM 2.5 Architecture

7.1 Overview

LFM 2.5 (January 2026) is Liquid AI's production model family, extending pretraining from 10T to 28T tokens on the LFM2 architecture (arXiv:2511.23404). It is designed for **on-device deployment** — smartphones, laptops, IoT devices, vehicles — with a memory footprint under 900 MB.

Model variants:

Model	Parameters	Focus
LFM2.5-1.2B-Base	1.2B	General pretrained base
LFM2.5-1.2B-Instruct	1.2B	Instruction following
LFM2.5-1.2B-Thinking	1.2B	Chain-of-thought reasoning
LFM2.5-1.2B-JP	1.2B	Japanese language
LFM2.5-VL-1.6B	1.6B	Vision-language
LFM2.5-Audio-1.5B	1.5B	Audio understanding

7.2 Hybrid Backbone: LIV Convolutions + GQA

The LFM2.5-1.2B architecture uses **16 layers** composed of:

- **10 double-gated LIV (Linear Input-Varying) convolution blocks**
- **6 Grouped-Query Attention (GQA) blocks**

This is an evolution of the StripedHyena pattern, with the Hyena-style convolutions replaced by the more efficient LIV convolutions.

7.3 Linear Input-Varying (LIV) Convolutions

An LIV system is a linear operator whose parameters are **functions of the input**:

$$y(t) = \int_0^t k(t-s;\, u(t))\, x(s)\, ds$$

where the convolution kernel k depends on the current input $u(t)$. In discrete form for a sequence:

$$y_t = \sum_{s=0}^{w-1} k_s\big(u_t\big)\, x_{t-s}$$

where w is the (short) kernel width and $k_s(u_t)$ is the s-th kernel coefficient, dynamically generated from the input at position t.

Intuition: A standard convolution uses a fixed filter that slides over the sequence. An LIV convolution uses a filter whose shape **changes at every position** based on what the input looks like there. This is the "liquid" principle applied to convolutions rather than to ODE time constants.

7.4 Double-Gated Structure

Each LIV convolution block uses two multiplicative gates:

$$z = \text{LIV-Conv}\big(\sigma_B(u) \odot x\big)$$
$$y = \sigma_C(u) \odot z$$

where:

- $\sigma_B(u)$: **Gate B** — modulates the input *before* the convolution
- $\sigma_C(u)$: **Gate C** — modulates the output *after* the convolution
- Both gates are learned functions of the input u
- $\odot$ is element-wise multiplication

Intuition: Gate B selects which input features are relevant before performing the convolution (input filtering). Gate C selects which output features to pass through (output filtering). Together, they give the block flexible, input-adaptive behavior without the quadratic cost of attention.

7.5 Short-Range vs. Long-Range

Unlike the original Hyena operator which uses implicit **long** convolutions (kernel length $\sim L$), LFM 2.5's LIV convolutions are **short-range** — they have a small local receptive field. Long-range dependencies are handled by the interleaved GQA blocks.

This design choice is driven by edge deployment: short convolutions are far more efficient on mobile CPUs and NPUs than long implicit convolutions.

7.6 STAR: Automated Architecture Search

The specific arrangement (which layers are convolutions vs. attention, kernel sizes, hidden dimensions, etc.) is found using **STAR** (Synthesis of Tailored Architectures), an evolutionary search framework presented at ICLR 2025 (Oral).

STAR encodes architectures as numerical "genomes" and uses evolutionary algorithms (recombination + mutation) to optimize for multiple objectives simultaneously:

- Prediction quality (perplexity)

- Model size (parameter count)
- Inference latency on target hardware
- Memory footprint (inference cache)

STAR-generated architectures achieve 13% parameter reduction and up to 90% inference cache reduction vs. standard Transformers, while maintaining quality.

7.7 Benchmarks (LFM2.5-1.2B-Instruct)

Benchmark	Score	Comparison
MMLU Pro	44.35	Beats Llama-3.2-1B, Gemma-3-1B
GPQA	38.89	Best in 1B class
IFEval	86.23	Instruction following
MATH-500 (Thinking)	88	Strong math reasoning
Decode speed (AMD CPU)	239 tok/s	Edge-optimized
Memory	$<$900 MB	Fits on phones

---

8. Nonlinearities in the LFM 2.5 Era

8.1 Historical Progression

The choice of nonlinearity has shifted significantly across the LNN/LFM lineage:

Era	Nonlinearity	Context
LTC (2021)	Sigmoid, tanh	Bounded activations for ODE stability
CfC (2022)	Sigmoid gating	Interpolation between learned states
S4/Liquid-S4	None explicit (linear SSM)	Nonlinearity via gating around SSM
StripedHyena	GELU	Standard Transformer-era choice
LFM 2.5 (2026)	**SwiGLU** (likely)	Current state-of-the-art for LLMs

8.2 SwiGLU: The Dominant Nonlinearity (Late 2025 / Early 2026)

SwiGLU (Swish-Gated Linear Unit), introduced by Shazeer (2020), is the standard activation in modern LLMs (LLaMA, PaLM, Mistral, etc.) and is the most likely choice in LFM 2.5's feedforward sublayers.

The SwiGLU feed-forward network replaces the standard two-layer FFN:

Standard FFN:

$$\mathrm{FFN}(x) = \mathrm{GELU}(x\mathbf{W}_1 + b_1)\,\mathbf{W}_2 + b_2$$

SwiGLU FFN:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \big[\mathrm{SiLU}(x\mathbf{W}) \odot (x\mathbf{V})\big]\,\mathbf{W}_2$$

where:

- $\mathrm{SiLU}(z) = z \cdot \sigma(z)$ is the Sigmoid Linear Unit (also called Swish with $\beta = 1$)
- $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid
- $\mathbf{W}, \mathbf{V} \in \mathbb{R}^{d \times d_{ff}}$ are two parallel linear projections
- $\mathbf{W}_2 \in \mathbb{R}^{d_{ff} \times d}$ is the output projection
- $\odot$ is element-wise multiplication

Intuition: SwiGLU splits the FFN into two paths. One path ($x\mathbf{V}$) computes a candidate representation. The other path ($\mathrm{SiLU}(x\mathbf{W})$) computes a smooth gate that controls how much of each feature dimension passes through. This is richer than a simple pointwise nonlinearity because the gate and the candidate interact multiplicatively, enabling more expressive feature selection.

8.3 Additional Nonlinearity from Double Gating

In LFM 2.5, the LIV convolution blocks provide **additional nonlinear expressivity** beyond the FFN activation:

- Gate B applies a learned nonlinear transformation before convolution
- Gate C applies another after convolution
- These gates are themselves parameterized by neural networks with SiLU or similar activations

This means the overall nonlinearity of an LIV block is a composition of multiple gating operations, each input-dependent. The total transformation is significantly more expressive than a single activation function.

---

9. Recurrent, Feedforward, or Transformer?

9.1 The Original LNNs: Purely Recurrent

LTC networks and CfC networks are **recurrent** models. They process sequences one step at a time, maintaining a hidden state that evolves according to an ODE (LTC) or its closed-form approximation (CfC). They cannot be parallelized across the sequence dimension during training.

Use cases: Robotics, autonomous driving, time-series with irregular sampling, applications requiring tiny models (19-neuron NCP controllers).

9.2 S4 and Liquid-S4: Dual-Mode

S4 and Liquid-S4 can operate in **two modes**:

- **Convolutional mode** (training): The entire sequence is processed in parallel as a 1D convolution. This is $O(L \log L)$ via FFT.
- **Recurrent mode** (inference): The model processes tokens one at a time with a fixed state size. This is efficient for autoregressive generation.

This dual nature is the bridge between the recurrent world (LTC, CfC) and the feedforward world (convolutions, Transformers).

9.3 LFM 2.5: Primarily Feedforward at Inference

LFM 2.5 is **not a recurrent network** in the traditional sense. At inference:

- The **LIV convolution blocks** are feedforward — they apply short convolutions with input-dependent kernels. No hidden state is maintained between tokens beyond the convolutional window.
- The **GQA attention blocks** are feedforward with a KV cache, following standard Transformer practice.

The "liquid" principle survives in the input-dependence of the LIV kernels and gates, but the sequential ODE integration of early LNNs is completely gone.

9.4 Relationship to Transformers

LFM 2.5 is **not** "a Transformer enhanced with liquid layers." It is better described as a **convolution-dominant hybrid** that uses selective attention:

- The majority of layers (10/16) are **not** attention — they are LIV convolutions
- Attention is used sparingly (6/16 layers) for global context
- The architecture is designed by STAR to minimize attention usage while maintaining quality

This is philosophically different from approaches that add liquid layers to a Transformer backbone. LFM 2.5 starts from convolutions and adds just enough attention.

9.5 Summary Table

Architecture	Mode	Sequence Handling	Attention?
LTC	Recurrent	ODE integration	No
CfC	Recurrent	Closed-form step	No
Liquid-S4	Recurrent / Conv	Dual mode	No
StripedHyena	Feedforward	Long conv + GQA	Sparse
LFM 2.5	**Feedforward**	**Short LIV conv + GQA**	**Sparse (6/16)**

10. Key Equations Summary

For quick reference, the core equations of the LNN-to-LFM evolution:

LTC ODE:

$$\frac{dx}{dt} = -\big[w + f_\theta(I)\big]\, x + A\big[w + f_\theta(I)\big]$$

CfC Closed-Form:

$$x(t) = \big(x(0) - A\big)\exp\!\big(-[1 + f_\theta(I)]\, t\big) + A$$

CfC Sigmoid Interpolation:

$$x(t) = \sigma(g_\theta) \odot f_1(I, x_0) + (1 - \sigma(g_\theta)) \odot f_2(I, x_0)$$

S4 Continuous SSM:

$$x'(t) = \mathbf{A}\, x(t) + \mathbf{B}\, u(t), \qquad y(t) = \mathbf{C}\, x(t)$$

Liquid-S4 (Input-Dependent):

$$x'(t) = \mathbf{A}(u)\, x(t) + \mathbf{B}\, u(t)$$

Hyena Recurrence:

$$y_i = (y_{i-1} * h_i) \odot x_i, \quad i = 1, \ldots, N$$

LIV Convolution (LFM 2.5):

$$y_t = \sum_{s=0}^{w-1} k_s(u_t)\, x_{t-s}$$

Double-Gated LIV Block:

$$z = \text{LIV-Conv}(\sigma_B(u) \odot x), \qquad y = \sigma_C(u) \odot z$$

SwiGLU FFN:

$$\text{FFN}_{\text{SwiGLU}}(x) = \big[\text{SiLU}(x\mathbf{W}) \odot (x\mathbf{V})\big]\mathbf{W}_2$$

References

1. Lechner, M., Hasani, R., et al. "Neural Circuit Policies Enabling Auditable Autonomy." *Nature Machine Intelligence*, 2020.
2. Hasani, R., Lechner, M., Amini, A., Rus, D., Grosu, R. "Liquid Time-constant Networks." *AAAI*, 2021.
3. Hasani, R., Lechner, M., et al. "Closed-form Continuous-time Neural Networks." *Nature Machine Intelligence*, 2022.
4. Gu, A., Goel, K., R'e, C. "Efficiently Modeling Long Sequences with Structured State Spaces." *ICLR*, 2022.
5. Hasani, R., Lechner, M., Wang, T.-H., Chahine, M., Amini, A., Rus, D. "Liquid Structural State-Space Models." *ICLR*, 2023.
6. Gu, A., Dao, T. "Mamba: Linear-Time Sequence Modeling with Selective State Spaces." *arXiv:2312.00752*, 2023.
7. Poli, M., Massaroli, S., et al. "Hyena Hierarchy: Towards Larger Convolutional Language Models." *ICML*, 2023.
8. Liquid AI. "LFM2 Technical Report." *arXiv:2511.23404*, November 2025.
9. Liquid AI. "STAR: Synthesis of Tailored Architectures." *ICLR*, 2025 (Oral).
10. Liquid AI. "Liquid Foundation Models 2.5." January 2026. https://www.liquid.ai/liquid-foundation-models
11. Shazeer, N. "GLU Variants Improve Transformer." *arXiv:2002.05202*, 2020.